

The Control Library

Ball Balancing Platform — student guide

Ball Balancing Platform project

Library version 1.0 — October 6, 2026

Abstract

This guide covers everything you can use in `control.cpp`: what the library gives you about the ball, how your plate angles reach the servos, the exact equations behind the PID helper and the filtered velocity, and what the delays in the Setup tab do. It then uses a simple model of the platform to design a PD controller, and the gains of the starter code fall out of that design. You do not need to read the physics model to use this guide, but it is there when you want the details [3].

Contents

1	The control loop	1
2	Coordinates and sign conventions	2
3	What you get: the Ball	2
3.1	How the position is measured	2
3.2	Velocity	3
4	What you set: the Platform	3
5	Helpers	3
5.1	PID	3
5.2	log, clamp, printf	3
6	Timing and the Setup effects	4
7	A model for design	4
7.1	Why P alone does not work	4
7.2	PD design by pole placement	4
7.3	The cost of delay	5
7.4	The integral term and the dead zone	5
7.5	Feedforward for moving targets	5
8	The starter code	5
9	Things to try	5

1 The control loop

Every time the camera delivers a frame, the library does the following, in this order:

1. It calls your `track()` (see the vision library guide), which returns the ball's position in pixels.
2. It converts that pixel position to millimetres on the plate. This accounts for the lens distortion and for refraction through the plate. It also adds the ball to the history and updates the filtered velocity.
3. It calls your `control()`:

```
void control(const Ball& ball, const Target& target, Platform& platform);
```

4. It limits your angles, converts them into the three servo angles (inverse kinematics), and sends them to the servos. This happens after the modelled FireBeetle run time and any motor delay you set (section 6).

So `control()` runs once per camera frame: 30 times a second by default, or fewer if you cap the frame rate.

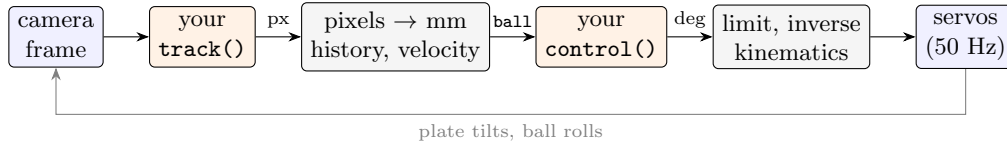


Figure 1: One pass of the loop. Orange boxes are your code; grey boxes are the library.

2 Coordinates and sign conventions

- **Positions** are in millimetres on the plate surface. The origin is at the centre of the plate, and x and y are the plate's axes (fig. 2). The plate has a radius of 100 mm; a 20 mm ball falls off once its centre is beyond 100 mm.
- **Angles** are in degrees. `platform.angleX` is the angle the plate surface makes with the horizontal along x . A *positive* value lifts the $+x$ edge, so the ball rolls towards $-x$. The same holds for `angleY`.
- **Time** is in seconds.

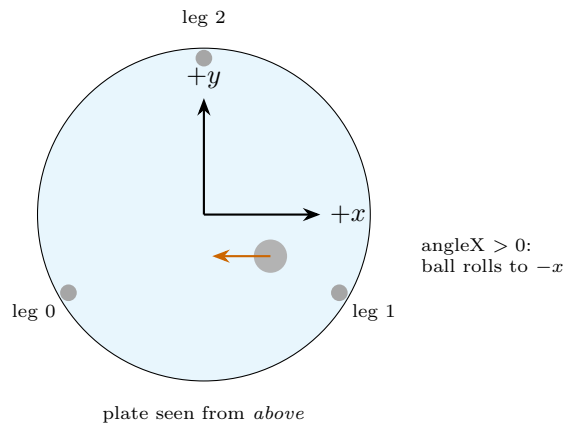


Figure 2: Plate coordinates seen from above. Leg 2 is on the $+y$ side.

3 What you get: the Ball

<code>ball.found</code>	<code>false</code> when <code>track()</code> did not find the ball this frame. The other fields then still hold the last good values.
<code>ball.x</code> , <code>ball.y</code>	Latest measured position [mm].
<code>ball.vx</code> , <code>ball.vy</code>	Filtered velocity [mm/s], see section 3.2.
<code>ball.dt</code>	Time since the previous <code>control()</code> call [s], normally 1/fps.
<code>ball.time</code>	Current time [s].
<code>ball.history(k)</code>	The k -th previous measurement, with <code>.x</code> , <code>.y</code> , <code>.t</code> . $k = 0$ is the newest; up to 64 are kept.
<code>ball.historySize()</code>	How many samples are stored.

3.1 How the position is measured

Your tracker returns pixels. The library traces that pixel back out of the camera (undoing the lens distortion), through the transparent plate (accounting for refraction), to the height of the ball's centre. Two consequences:

- The library uses the plate angle you *commanded*, not the actual angle, because the firmware cannot know the actual angle. While the plate is still moving, this adds a small error.
- The measurement has noise from the camera image, a small systematic offset from perspective (about 1 mm near the plate edge), and any artificial position noise you set.

3.2 Velocity

The velocity is estimated from consecutive positions and smoothed with a first-order (exponential) filter:

$$v_k = \alpha v_{k-1} + (1 - \alpha) \frac{x_k - x_{k-1}}{t_k - t_{k-1}}, \quad (1)$$

where α is the *velocity filter* in Setup > Control (default 0.5). A raw difference ($\alpha = 0$) amplifies measurement noise: an error of 0.3 mm at 30 fps becomes velocity noise of order 10 mm/s. Smoothing reduces the noise but adds lag, about $\tau = -\Delta t / \ln \alpha$, which is 48 ms for $\alpha = 0.5$ at 30 fps. If you prefer to build your own estimator, use `ball.history()`.

4 What you set: the Platform

Set `platform.angleX` and `platform.angleY` in degrees. Then:

1. The library clamps them to $\pm \text{Max plate angle}$ (Setup, default 8°).
2. It turns them into a plate orientation. The plate normal is $\mathbf{n} \propto (-\tan \theta_x, -\tan \theta_y, 1)$, so the two angles are independent and the order you set them in does not matter. The plate height stays fixed.
3. It solves the inverse kinematics for the three crank angles and sends them as servo pulses. The servos take a new pulse every 20 ms, then move at their limited speed.

If you ask for something out of reach, the command is ignored and a warning is shown.

Target. `target.x`, `target.y` [mm] is where the ball should go. It is set in the Setup tab: a fixed point, or a circle, square or figure eight. You can also move it live from the Simulation tab.

5 Helpers

5.1 PID

```
PID pid(kp, ki, kd);           // create once, outside control()
float u = pid.update(error, ball.dt);
pid.setDerivativeFilter(0.03f); // optional: low-pass the D term (seconds)
pid.setIntegralLimit(50.0f);    // optional: anti-windup
pid.setOutputLimit(8.0f);       // optional: clamp the output
pid.reset();                    // forget the integral and history
```

Create PID objects *outside* the function, so they keep their state between calls. Each `update(e, dt)` computes exactly:

$$I_k = \text{clamp}(I_{k-1} + e_k \Delta t, \pm I_{\max}), \quad (2)$$

$$d_k = \frac{e_k - e_{k-1}}{\Delta t} \quad (0 \text{ on the first call}), \quad D_k = D_{k-1} + \frac{\Delta t}{\tau_d + \Delta t} (d_k - D_{k-1}), \quad (3)$$

$$u_k = \text{clamp}(k_p e_k + k_i I_k + k_d D_k, \pm u_{\max}), \quad (4)$$

where $D_k = d_k$ when no derivative filter is set ($\tau_d = 0$), and a limit of 0 means no limit. The three terms of the last update are in `pid.lastP`, `lastI` and `lastD`. Logging them is a good way to see which term is doing what.

5.2 log, clamp, printf

`log("name", value)` plots any value live in the Simulation tab, up to six series. `clamp(v, lo, hi)` limits a value. `printf` prints to the output console; don't call it every frame, because it is slow.

6 Timing and the Setup effects

The ball sees your command later than you might think. Between the light leaving the ball and the plate moving there are:

Stage	Default	Setup control
Exposure + rolling readout + transfer	≈ 24 ms	Camera: exposure
Extra sensor delay	0	Artificial effects: sensor delay
Your code on the FireBeetle	a few ms	measured $\times$ slowdown factor
Extra motor delay	0	Artificial effects: motor delay
Waiting for the next servo pulse	0–20 ms	—
Servo reaching the angle	~ 30 – 100 ms	— (Simulated hardware: servos)

The *frame-rate cap* drops frames, so `control()` runs less often and `ball.dt` grows. *Position noise* adds random error to `ball.x` and `ball.y`. These effects exist so you can see how each kind of non-ideality hurts a controller, and how to design around it.

7 A model for design

For small angles, a ball rolling without slipping on a tilted plate accelerates at

$$\ddot{x} = -\frac{g}{1+k} \sin \theta_x \approx -G \theta_x, \quad G = \frac{g}{1+k} \cdot \frac{\pi}{180}, \quad (5)$$

with θ_x in degrees. The factor $1+k$ is the rolling inertia: $k = \frac{2}{5}$ for a solid ball, about $\frac{2}{3}$ for a thin-walled hollow ball [3]. Numerically:

Ball	k	G [mm/s ² per degree]
solid (steel, glass, plastic)	2/5	122
hollow, thin wall	$\approx 2/3$	103

The x and y axes are (almost) independent, so you can design one axis and copy it. In the Laplace domain each axis is a double integrator with a delay T :

$$\frac{X(s)}{\Theta(s)} = -\frac{G}{s^2} e^{-sT}. \quad (6)$$

The full model also includes servo dynamics, friction and parasitic motion, and the simulator uses all of it [3]. This simple model is what you design with.

7.1 Why P alone does not work

With $\theta_x = -k_p e$ and $e = x_t - x$ (a fixed target), (5) gives $\ddot{x} + Gk_p x = Gk_p x_t$. That is an undamped oscillator: the ball swings through the target forever. Any delay makes it grow. You need damping, which is the D term.

7.2 PD design by pole placement

With $\theta_x = -(k_p e + k_d \dot{e})$ and $\dot{e} = -\dot{x}$:

$$\ddot{x} + Gk_d \dot{x} + Gk_p x = Gk_p x_t \implies \omega_n^2 = Gk_p, \quad 2\zeta\omega_n = Gk_d. \quad (7)$$

Choose the natural frequency ω_n (speed) and the damping ζ (overshoot), then

$$k_p = \frac{\omega_n^2}{G}, \quad k_d = \frac{2\zeta\omega_n}{G}. \quad (8)$$

Example. With $\omega_n = 3.45$ rad/s and $\zeta = 0.85$, the gains are $k_p = 0.097$ and $k_d = 0.048$, and the settling time is about $4/(\zeta\omega_n) \approx 1.4$ s. The starter code uses $k_p = 0.09$, $k_d = 0.045$, which is this design rounded.

7.3 The cost of delay

A delay T removes phase ωT at frequency ω . Around the crossover frequency $\omega_c \approx \omega_n$, the phase margin drops by

$$\Delta\varphi \approx \omega_c T \text{ rad} = 57.3 \omega_c T \text{ degrees.} \quad (9)$$

With the default $T \approx 60$ ms and $\omega_c = 3.5$ rad/s, that is 12° . Add 150 ms of sensor delay and you lose another 30° , and the starter controller starts to oscillate. A slower design (smaller ω_n) tolerates more delay. This trade-off is the central lesson of the Setup effects [2, 4].

7.4 The integral term and the dead zone

Rolling resistance gives the ball a dead zone. On a plate tilted less than about $\arctan c_{rr}$ (typically under 0.1°) it simply stays where it is. A PD controller then leaves a small steady error. An integral term removes it, but integrates during large moves too; limit it with `setIntegralLimit`. PID tuning in general is covered by Åström and Hägglund [1].

7.5 Feedforward for moving targets

To follow a moving target $x_t(t)$, the ball needs acceleration $\ddot{x}_t$. Adding the angle that produces it, $\theta_{ff} = -\ddot{x}_t/G$, lets the feedback handle only the errors. For a circle of radius R and period P , $\ddot{x}_t$ has magnitude $R(2\pi/P)^2$. That is 25 mm/s^2 , or 0.2° , for the default 40 mm, 8 s circle.

8 The starter code

```
#include <bbp/bbp.hpp>
using namespace bbp;

PID pidX(0.09f, 0.0f, 0.045f);
PID pidY(0.09f, 0.0f, 0.045f);

void control(const Ball& ball, const Target& target, Platform& platform) {
    if (!ball.found) { // lost the ball: level the plate
        platform.angleX = 0;
        platform.angleY = 0;
        return;
    }
    pidX.setDerivativeFilter(0.03f);
    pidY.setDerivativeFilter(0.03f);
    const float errorX = target.x - ball.x;
    const float errorY = target.y - ball.y;
    // ball too far to +x (error < 0) -> lift the +x edge (angle > 0)
    platform.angleX = -pidX.update(errorX, ball.dt);
    platform.angleY = -pidY.update(errorY, ball.dt);
    log("error x [mm]", errorX);
    log("error y [mm]", errorY);
}
```

9 Things to try

1. Set $k_d = 0$. Watch the ball oscillate, and explain why with the P-only analysis above.
2. Add 100 ms, then 200 ms, of sensor delay. Re-tune with a smaller ω_n .
3. Replace the PID's derivative with `ball.vx`. How does the velocity filter change the result?
4. Set the target to a circle. Add the feedforward term and compare the tracking error.
5. Switch to a hollow ball (Simulated hardware). Which gain must change, and by how much?

References

- [1] Karl Johan Åström and Tore Hägglund. *PID Controllers: Theory, Design, and Tuning*. Instrument Society of America, 2nd edition, 1995.
- [2] Karl Johan Åström and Richard M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.
- [3] Ball Balancing Platform project. Physics and modelling of the ball balancing platform simulator. Model specification, available from the app's Documentation menu, 2026.
- [4] Gene F. Franklin, J. David Powell, and Abbas Emami-Naeini. *Feedback Control of Dynamic Systems*. Pearson, 7th edition, 2015.